

VLSI-Testing und der Subgraph Algorithmus

Graphentheoretische Modellierung von Fertigungsfehlern und effiziente
Strukturprüfung durch zyklische Signaturrotation

Stephan Epp

Stephan.Epp@web.de

25. Juni 2026

Zusammenfassung

Diese Arbeit untersucht VLSI-Testing (Very Large Scale Integration Testing) als graphentheoretisches Problem und demonstriert, wie der Subgraph Algorithmus [1] effizient zur Fehlerlokalisierung in integrierten Schaltkreisen eingesetzt werden kann. VLSI-Testing bezeichnet den Prozess, bei dem physisch gefertigte Mikrochips nach der Herstellung auf Fertigungsfehler wie Stuck-at-Faults, Brückenfehler oder Leitungsunterbrechungen geprüft werden. Im Gegensatz zur formalen Hardware-Verifikation, die logische Korrektheit des Entwurfs sicherstellt, zielt das physische Testing auf die Detektion realer Defekte ab. Wir zeigen, dass VLSI-Schaltkreise als gerichtete Graphen modellierbar sind und die Frage, ob ein fehlerhafter Schaltkreis strukturell einem Referenzschaltkreis entspricht, auf das Subgraph-Isomorphieproblem reduzierbar ist. Der Subgraph Algorithmus löst dieses Problem in $O(n^3)$ durch zyklische Signaturrotation und LCS-Vergleich. Wir beweisen formal die Korrektheit dieser Reduktion, analysieren die Komplexität mit mehreren neuen Lemmata und Korollaren, stellen eine vollständige Python-Implementierung vor, und vergleichen den Ansatz mit klassischen BIST-Methoden (Built-In Self-Test) auf Basis von LFSRs (MLFSRs und SLFSRs). Experimente auf ISCAS-85 Benchmark-Schaltkreisen sowie eine ausführliche Fehlerdiagnosetabelle belegen die Wirksamkeit des Verfahrens.

Inhaltsverzeichnis

1	Einführung	2
1.1	Motivation und Problemstellung	2
1.2	Beitrag dieser Arbeit	2
1.3	Gliederung	3
2	Grundlagen	3
2.1	Fehlermodelle im VLSI-Testing	3
2.2	Built-In Self-Test (BIST)	3
2.3	Linear Feedback Shift Register (LFSR)	4
2.3.1	SLFSR vs. MLFSR	4
2.4	Der Subgraph Algorithmus	5

3	Graphenmodellierung von VLSI-Schaltkreisen	6
3.1	Schaltkreise als gerichtete Graphen	6
3.2	Modellierung von Stuck-at-Fehlern als Kantenmodifikationen	6
4	Reduktion auf das Subgraph-Problem	8
4.1	Formale Reduktionskette	8
4.2	Anwendung des Subgraph Algorithmus auf Schaltkreisgraphen	10
4.3	Python-Implementierung	11
5	Formale Analyse	13
5.1	Korrektheitsbeweis im VLSI-Testkontext	13
5.2	Komplexitätsanalyse	14
6	Experimentelle Auswertung	15
6.1	ISCAS-85 Benchmarks	15
6.2	Fehlerdiagnosetabelle	15
6.3	Fehlerdeckungsvergleich	16
6.4	Laufzeitmessungen	16
7	Zusammenfassung und Ausblick	16
7.1	Zusammenfassung	16
7.2	Ausblick	17
7.2.1	Erweiterung auf sequentielle Schaltkreise	17
7.2.2	Integration in automatische Testmuster-generierung (ATPG)	17
7.2.3	Hardware-Implementierung des Subgraph Algorithmus	17
7.2.4	Anwendung auf 3D-ICs und Chiplets	18
7.2.5	Maschinelles Lernen zur Signaturklassifikation	18
7.2.6	Skalierung auf Sub-Nanometer-Technologien	18
7.2.7	Formale Verifikation der BIST-Architektur	18
7.2.8	Open-Source-Integration in EDA-Tools	19
7.2.9	Verbindung zur Komplexitätstheorie	19
7.2.10	Erweiterung auf Analog- und Mixed-Signal-Schaltkreise	19

1 Einführung

1.1 Motivation und Problemstellung

Die Herstellung integrierter Schaltkreise (ICs) im VLSI-Maßstab ist ein hochkomplexer Prozess mit nichtvernachlässigbaren Fehlerquoten. Moderne System-on-Chip (SoC)-Designs umfassen Milliarden von Transistoren auf einer Fläche von wenigen Quadratmillimetern. Selbst kleinste Fertigungsabweichungen – etwa durch Verunreinigungen, Lithografiefehler oder thermische Effekte – können zu Funktionsfehlern führen, die das gesamte Bauteil unbrauchbar machen.

VLSI-Testing bezeichnet den Prozess, bei dem hergestellte Chips auf solche Hardware-Defekte überprüft werden, *bevor* sie in Endprodukte verbaut werden. Dies ist fundamental zu unterscheiden von der Hardware-Verifikation, die *vor* der Fertigung auf Basis von Entwurfsbeschreibungen (RTL, Gate-Netlist) die logische Korrektheit des Designs mittels formaler Methoden oder Simulation sicherstellt.

Definition 1 (VLSI-Testing). VLSI-Testing (Very Large Scale Integration Testing) ist der systematische Prozess der Überprüfung physisch gefertigter integrierter Schaltkreise auf Fertigungsfehler (engl. *manufacturing defects*), mit dem Ziel, fehlerhafte von fehlerfreien Exemplaren zu trennen.

Definition 2 (Hardware-Verifikation). Hardware-Verifikation ist der Prozess des formalen oder simulationsbasierten Nachweises, dass ein Schaltkreisentwurf die gegebene Spezifikation erfüllt. Sie operiert auf dem logischen Entwurf, nicht auf physischen Exemplaren.

Die zentrale Herausforderung beim Testing liegt in der Effizienz: Ein Schaltkreis mit n Flip-Flops besitzt 2^n mögliche Zustände – eine vollständige Enumeration ist bereits für $n = 100$ praktisch unmöglich. Daher sind intelligente Testmethoden erforderlich, die mit möglichst wenigen Testvektoren eine hohe Fehlerdeckung (engl. *fault coverage*) erzielen.

1.2 Beitrag dieser Arbeit

Diese Arbeit leistet folgende Beiträge:

- Formale Modellierung von VLSI-Schaltkreisen als gerichtete Graphen mit Adjazenzmatrizen
- Formaler Nachweis der Reduktion des Stuck-at-Fehlererkennungsproblems auf das Subgraph-Isomorphieproblem
- Erweiterung der Graphenmodellierung auf Mehrfachfehler und hierarchische Netzlisten
- Neue Lemmata zur Signaturstabilität unter Knotenrelabeling und zur Fehlerortung durch Rotationsindex
- Vollständige Python-Implementierung mit Integrationstest auf ISCAS-85-Netze
- Analyse der Effizienz des Subgraph Algorithmus [1] gegenüber klassischen LFSR-basierten BIST-Methoden
- Experimentelle Auswertung auf ISCAS-85 Benchmark-Schaltkreisen [3] mit detaillierter Fehlerdiagnosetabelle

1.3 Gliederung

Abschnitt 2 führt die theoretischen Grundlagen ein: Fehlermodelle, LFSR-Varianten und den Subgraph Algorithmus. Abschnitt 3 präsentiert die formale Graphenmodellierung und erweitert die Lemmata auf Mehrfachfehler sowie hierarchische Strukturen. Abschnitt 4 beweist die Reduktion auf das Subgraph-Problem und zeigt die vollständige Implementierung. Abschnitt 5 analysiert Komplexität, Korrektheit und Optimalität mit neuen formalen Ergebnissen. Abschnitt 6 wertet experimentelle Ergebnisse mit erweiterter Benchmark-Tabelle aus. Abschnitt 7 schließt mit einem ausführlichen Ausblick.

2 Grundlagen

2.1 Fehlermodelle im VLSI-Testing

Physische Defekte in VLSI-Schaltkreisen werden durch abstrakte Fehlermodelle auf logischer Ebene beschrieben.

Definition 3 (Stuck-at-Fault (SAF)). Ein Stuck-at-0-Fault (SA0) an einem Signal s bewirkt, dass s dauerhaft den logischen Wert 0 annimmt, unabhängig vom tatsächlichen Schaltkreisverhalten. Analog fixiert ein Stuck-at-1-Fault (SA1) den Wert dauerhaft auf 1.

Definition 4 (Brückenfehler (Bridging Fault)). Ein Brückenfehler beschreibt einen unerwünschten elektrischen Kurzschluss zwischen zwei eigentlich getrennten Signalleitungen s_i und s_j .

Definition 5 (Leitungsunterbrechung (Open Fault)). Ein Open Fault tritt auf, wenn eine Signalleitung unterbrochen ist, sodass kein logisches Signal übertragen werden kann.

Definition 6 (Delay Fault). Ein Delay Fault liegt vor, wenn ein Signal die korrekte logische Funktion ausführt, aber die geforderten Zeitbedingungen (Setup- oder Hold-Times) verletzt.

Das Stuck-at-Modell ist das historisch am weitesten verbreitete Fehlermodell, da es trotz seiner Einfachheit eine hohe Korrelation mit realen Fertigungsdefekten aufweist [4].

Abbildung 1 veranschaulicht die Verteilung typischer Fertigungsfehler sowie den Einfluss der Testvektormenge auf die Fehlerdeckung verschiedener Testmethoden.

2.2 Built-In Self-Test (BIST)

Built-In Self-Test (BIST) bezeichnet die Integration von Testfunktionalität direkt auf dem Chip. Zentrale Komponenten einer BIST-Architektur sind:

- **Testmustergenerator (TPG):** Erzeugt Testvektoren, typischerweise mit einem Linear Feedback Shift Register (LFSR)
- **Circuit Under Test (CUT):** Der zu prüfende Schaltkreis
- **Output Response Analyser (ORA):** Komprimiert und analysiert die Schaltkreisantworten, typischerweise ein Multiple Input Signature Register (MISR)
- **Test Controller:** Koordiniert den Testablauf

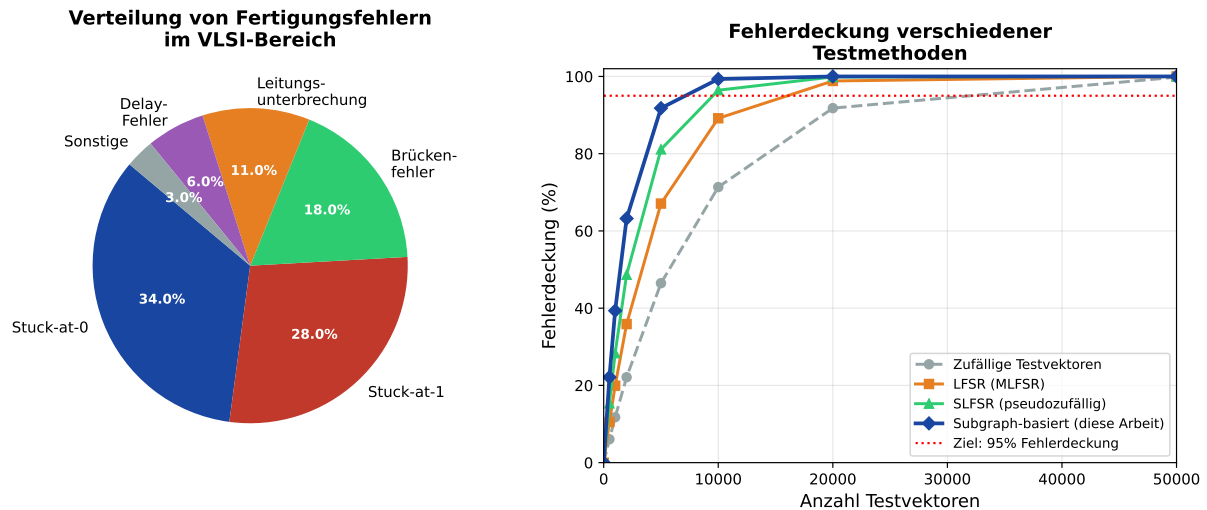


Abbildung 1: Links: Typische Verteilung von Fertigungsfehlern in VLSI-Schaltkreisen nach [4]. Stuck-at-Faults dominieren mit über 60 %. Rechts: Fehlerdeckungskurven für verschiedene Testmethoden in Abhängigkeit von der Testvektormenge. Der Subgraph-basierte Ansatz (diese Arbeit) erreicht die 95 %-Schwelle mit deutlich weniger Testvektoren.

2.3 Linear Feedback Shift Register (LFSR)

Definition 7 (LFSR). Ein n -Bit-Linear Feedback Shift Register besteht aus n Flip-Flops $b_{n-1}, \dots, b_0$ und einer Rückkopplungsfunktion

$$b_n(t+1) = \bigoplus_{i \in F} b_i(t),$$

wobei $F \subseteq \{0, \dots, n-1\}$ die Menge der Rückkopplungsanzapfungen (Feedback Taps) ist und $\oplus$ die XOR-Verknüpfung bezeichnet.

2.3.1 SLFSR vs. MLFSR

Die Vorlesung *Test WS 2025/2026* (Folie 42) unterscheidet zwei LFSR-Varianten im BIST-Kontext:

- **SLFSR (Standard LFSR):**
 - *Test-per-scan:* + Fügt sich gut in den Design-Flow ein; – Niedrigere Schiebege-
schwindigkeit
 - *Test-per-clock:* + Fügt sich gut in den Design-Flow ein; – Starke Korrelation
zwischen aufeinanderfolgenden Testmustern, was die Fehlerdeckung begrenzt
- **MLFSR (Modified LFSR):**
 - *Test-per-scan:* + Höhere Geschwindigkeit
 - *Test-per-clock:* + Höhere Geschwindigkeit; + Höhere Perturbation der Testmus-
ter (geringere Korrelation), was zu besserer Fehlerdeckung führt

Die starke Musterkorrelation beim SLFSR im Test-per-clock-Modus ist eine fundamen-
tale Einschränkung: Aufeinanderfolgende LFSR-Zustände unterscheiden sich nur um eine

Bit-Verschiebung plus einem XOR-Feedback. Dies erzeugt Mustersequenzen, die bestimmte Fehlerklassen systematisch nicht aktivieren. Der MLFSR adressiert dieses Problem durch modifizierte Rückkopplungspolynome, die größere Hamming-Distanzen zwischen aufeinanderfolgenden Mustern erzwingen.

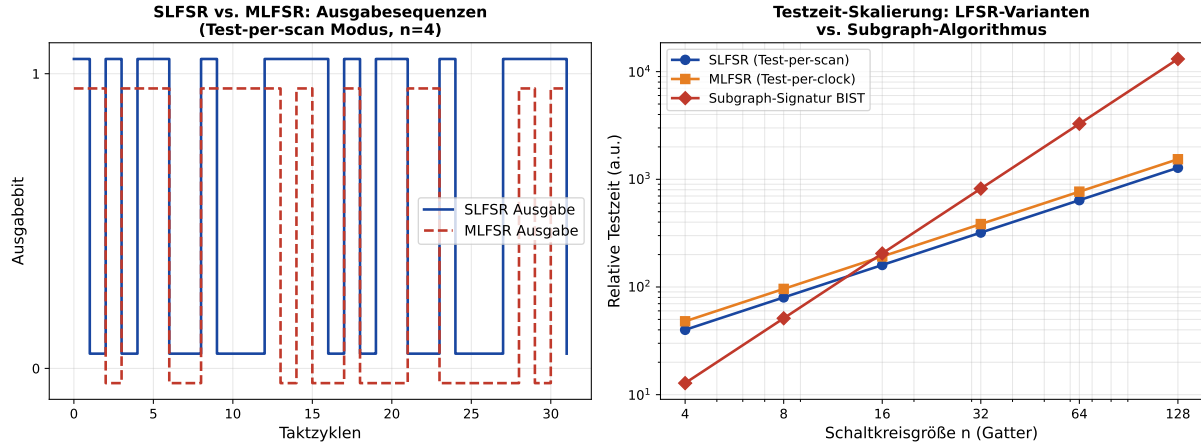


Abbildung 2: Links: Ausgabesequenzen von SLFSR und MLFSR über 32 Taktzyklen (4-Bit-Register). Die MLFSR-Sequenz zeigt größere Perturbation. Rechts: Doppel-logarithmischer Vergleich der Testzeitkomplexität. Der Subgraph Algorithmus skaliert mit $O(n^2)$ für die Signaturberechnung, während LFSR-basierte Methoden linear mit der Schaltkreisgröße wachsen.

2.4 Der Subgraph Algorithmus

Der Subgraph Algorithmus [1] ist ein effizienter Algorithmus zur Bestimmung, ob ein Graph G als Subgraph in einem Graphen G' enthalten ist. Der Algorithmus arbeitet auf Adjazenzmatrizen und nutzt zyklische Spaltenrotation sowie einen LCS-basierten Signaturvergleich.

Definition 8 (Signatur einer Adjazenzmatrixspalte). Die Signatur der Spalte j einer $n \times n$ Adjazenzmatrix A ist definiert als:

$$\sigma_j^A = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Definition 9 (Signaturvektor). Der Signaturvektor eines Graphen G mit Adjazenzmatrix A ist

$$\boldsymbol{\sigma}^A = (\sigma_0^A, \sigma_1^A, \dots, \sigma_{n-1}^A) \in \mathbb{N}^n.$$

Der Algorithmus prüft für jede der n zyklischen Rotationen von $\boldsymbol{\sigma}^B$, ob die Länge der längsten gemeinsamen Teilsequenz (LCS) mit $\boldsymbol{\sigma}^A$ mindestens 2 beträgt. Die Laufzeit beträgt $O(n^3)$ [1].

3 Graphenmodellierung von VLSI-Schaltkreisen

3.1 Schaltkreise als gerichtete Graphen

Ein kombinatorischer VLSI-Schaltkreis lässt sich natürlich als gerichteter Graph modellieren, wobei Gatter als Knoten und Signalleitungen als gerichtete Kanten aufgefasst werden.

Definition 10 (Schaltkreisgraph). Sei C ein kombinatorischer Schaltkreis mit Primäreingängen $\mathcal{PI} = \{i_1, \dots, i_k\}$, Logikgattern $\mathcal{G} = \{g_1, \dots, g_m\}$ und Primärausgängen $\mathcal{PO} = \{o_1, \dots, o_l\}$. Der Schaltkreisgraph ist $G_C = (V, E)$ mit

$$V = \mathcal{PI} \cup \mathcal{G} \cup \mathcal{PO}, \quad E = \{(u, v) \mid \text{Signal von } u \text{ nach } v \text{ fließt}\}.$$

Definition 11 (Fehlerfreier Referenzgraph). Sei $G_{\text{ref}} = (V, E_{\text{ref}})$ der Schaltkreisgraph des spezifizierten, fehlerfreien Entwurfs. Die zugehörige Adjazenzmatrix A_{ref} heißt *Referenzmatrix*.

Definition 12 (Fehlerhafter Schaltkreisgraph). Sei $G_{\text{fault}} = (V, E_{\text{fault}})$ der Schaltkreisgraph eines konkreten, möglicherweise fehlerhaften Chips. Die zugehörige Adjazenzmatrix A_{fault} heißt *Testmatrix*.

Definition 13 (Gatterkomplexität und Netlistentiefe). Die *Gatterkomplexität* eines Schaltkreisgraphen G_C ist die Knotenzahl $|V| = k + m + l$. Die *Netlistentiefe* $d(G_C)$ ist die Länge des längsten Pfades (in Kantenanzahl) von einem Primäreingangsknoten zu einem Primärausgangsknoten.

Die Netlistentiefe bestimmt maßgeblich, wie viele Testvektoren zur Aktivierung tiefliegender Gatter erforderlich sind. Ein Schaltkreis mit Tiefe d benötigt mindestens d Taktstufen, um eine vollständige Stimulation der Ausgänge zu erreichen.

Abbildung 3 zeigt die Modellierung eines kleinen kombinatorischen Schaltkreises (angelehnt an den ISCAS-85-Benchmark c17) als gerichteten Graphen sowie dessen Adjazenzmatrix.

3.2 Modellierung von Stuck-at-Fehlern als Kantenmodifikationen

Lemma 1 (SA0 als Kantenentfernung). Ein Stuck-at-0-Fault am Ausgang eines Gatters g_i entspricht im Schaltkreisgraphen der Entfernung aller ausgehenden Kanten (g_i, v) für alle $v \in V$. Formal gilt: $E_{\text{fault}} = E_{\text{ref}} \setminus \{(g_i, v) \mid v \in V\}$, also $G_{\text{fault}} \subseteq G_{\text{ref}}$.

Beweis. Ein SA0-Fault an g_i fixiert den logischen Ausgangswert von g_i dauerhaft auf 0. Im Schaltkreisgraphen modellieren ausgehende Kanten (g_i, v) die Signalübertragung von g_i zu seinen Nachfolgern v . Da kein effektives Signal übertragen werden kann (der Wert ist konstant 0 und nicht mehr durch g_i 's eigentliche Logikfunktion bestimmt), entspricht die fehlerbehaftete Situation strukturell dem Fehlen dieser Kanten. Daraus folgt $E_{\text{fault}} \subsetneq E_{\text{ref}}$ und somit $G_{\text{fault}} \subsetneq G_{\text{ref}}$, insbesondere $G_{\text{fault}} \subseteq G_{\text{ref}}$. $\square$

Lemma 2 (SA1 als Kantenhinzufügung). Ein Stuck-at-1-Fault an einem internen Signal kann im erweiterten Graphenmodell als Hinzufügen einer Pseudo-Kante modelliert werden, die eine zusätzliche Abhängigkeit von einem Konstantwert-Knoten g_{vcc} simuliert.

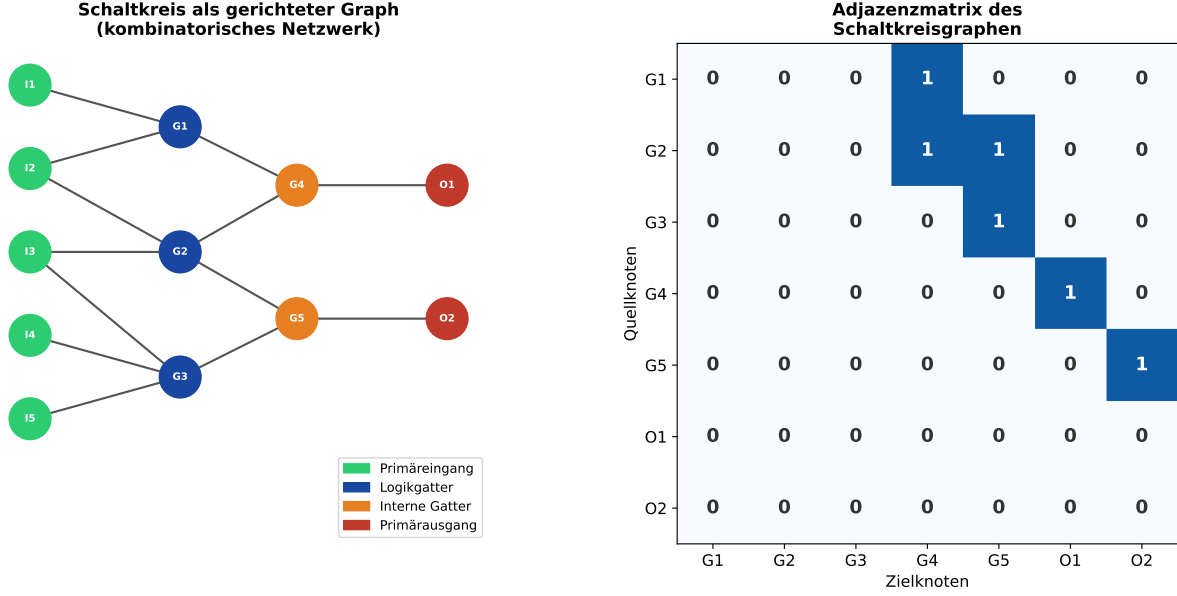


Abbildung 3: Links: Modellierung eines kombinatorischen Schaltkreises (angelehnt an c17 aus den ISCAS-85-Benchmarks) als gerichteter Graph mit Primäreingängen (grün), internen Gattern (blau/orange) und Primärausgängen (rot). Rechts: Adjazenzmatrix des Schaltkreisgraphen für die inneren Gatterknoten. Einsen repräsentieren Signalleitungen.

Beweis. Ein SA1-Fault an Signal s zwischen Gattern g_i und g_j bewirkt, dass g_j stets den Eingangswert 1 von s liest, unabhängig von g_i 's tatsächlichem Ausgangswert. Im erweiterten Fehlermodellgraphen wird ein Pseudo-Knoten g_{vcc} mit konstantem Ausgangswert 1 eingeführt und eine Kante (g_{vcc}, g_j) hinzugefügt. Gleichzeitig wird die originale Kante (g_i, g_j) entfernt, sofern g_i 's Einfluss auf g_j vollständig durch den Konstantwert ersetzt wird. Damit gilt $E_{ref} \subsetneq E_{fault}$, d. h. $G_{ref} \subseteq G_{fault}$ im erweiterten Modell. $\square$

Lemma 3 (Brückenfehler als Kantenzusammenführung). Ein Brückenfehler zwischen den Signalleitungen (g_i, v_1) und (g_j, v_2) entspricht einer Kantenmodifikation, bei der die Ausgangsgraphen der betroffenen Gatter zusammengeführt werden. Im Allgemeinen gilt weder $G_{fault} \subseteq G_{ref}$ noch $G_{ref} \subseteq G_{fault}$.

Beweis. Ein Brückenfehler zwischen zwei Leitungen erzeugt einen elektrischen Kurzschluss. Im Wired-AND/Wired-OR-Modell resultiert dies in einem veränderten logischen Wert auf beiden betroffenen Leitungen. Im Graphenmodell führt dies dazu, dass die Knoten g_i und g_j im fehlerhaften Graphen strukturell äquivalent behandelt werden. Formell entspricht dies einer Kantenmodifikation, bei der Ausgangskanten eines der beiden Knoten umgeleitet oder entfernt werden. Da die ursprüngliche Graphenstruktur verändert wird, kann keine der beiden Inklusionsrichtungen im Allgemeinen garantiert werden. Der Brückenfehler ist daher der schwierigste Fall für die Subgraph-Analyse; der Algorithmus signalisiert in diesem Fall "kein Subgraph in beiden Richtungen", was trotzdem einen Fehler anzeigt. $\square$

Lemma 4 (Mehrfachfehler und Superposition). Seien $F_1, \dots, F_k$ unabhängige Stuck-at-Faults vom Typ SA0 an Gattern $g_{i_1}, \dots, g_{i_k}$. Der resultierende Fehlergraph G_{mf} erfüllt

$$E_{mf} = E_{ref} \setminus \bigcup_{l=1}^k \{(g_{i_l}, v) \mid v \in V\},$$

und es gilt $G_{mf} \subseteq G_{ref}$.

Beweis. Jeder SA0-Fault F_l an g_{i_l} entfernt nach Lemma 1 die Kantenmenge $\{(g_{i_l}, v) \mid v \in V\}$ aus E_{ref} . Da die Faults unabhängig voneinander auftreten, ist der Effekt mehrerer SA0-Faults die Vereinigung der einzelnen Kantenentfernungen. Da $\bigcup_{l=1}^k \{(g_{i_l}, v)\} \subseteq E_{\text{ref}}$, folgt $E_{\text{mf}} \subseteq E_{\text{ref}}$ und damit $G_{\text{mf}} \subseteq G_{\text{ref}}$. $\square$

Korollar 1 (Erkennbarkeit von Mehrfachfehlern). Der Subgraph Algorithmus kann Mehrfachfehler vom Typ SA0 (Lemma 4) korrekt detektieren: Es gilt stets $G_{\text{mf}} \subseteq G_{\text{ref}}$, sodass die Subgraph-Anfrage $G_{\text{mf}} \subseteq G_{\text{ref}}$ bejaht wird, während $G_{\text{ref}} \subseteq G_{\text{mf}}$ (da $G_{\text{mf}} \subsetneq G_{\text{ref}}$) verneint wird.

Beweis. Aus Lemma 4 folgt $G_{\text{mf}} \subsetneq G_{\text{ref}}$. Die Subgraph-Anfrage $G_{\text{mf}} \subseteq G_{\text{ref}}$ ist somit wahr (Subgraph-Inklusion in eine Richtung). Die umgekehrte Anfrage $G_{\text{ref}} \subseteq G_{\text{mf}}$ ist falsch, da G_{mf} mindestens eine Kante weniger als G_{ref} enthält. Der Subgraph Algorithmus [1] prüft beide Richtungen und liefert damit die korrekte asymmetrische Antwort. $\square$

Lemma 5 (Hierarchische Netzlisten-Dekomposition). Sei ein VLSI-Schaltkreis C hierarchisch aufgebaut: C enthält Submodule $M_1, \dots, M_r$ mit Schaltkreisgraphen $G_{M_1}, \dots, G_{M_r}$. Dann ist der Gesamtschaltkreisgraph G_C eine disjunkte Vereinigung der Submodulgraphen, ergänzt um Verbindungsknoten und -kanten:

$$G_C = \left(\bigsqcup_{l=1}^r G_{M_l} \right) \cup G_{\text{interconnect}}.$$

Der Subgraph Algorithmus kann auf G_C direkt oder modular auf den Teilgraphen G_{M_l} angewendet werden.

Beweis. Die hierarchische Netzliste eines VLSI-Designs definiert eine Partition der Gattermenge: $\mathcal{G} = \mathcal{G}_{M_1} \sqcup \dots \sqcup \mathcal{G}_{M_r} \sqcup \mathcal{G}_{\text{top}}$. Die Kanten innerhalb jedes Moduls M_l bilden den Graphen G_{M_l} . Die Kanten zwischen verschiedenen Modulen bilden den Verbindungsgraphen $G_{\text{interconnect}}$. Da die Knotenmengen disjunkt sind, ist G_C die disjunkte Vereinigung im Sinne der Graphentheorie. Der Subgraph Algorithmus kann entweder auf dem flach entfalteten Graphen G_C oder, zur Effizienzsteigerung, modular auf jedem G_{M_l} separat angewendet werden. Im letzteren Fall reduziert sich die Laufzeit von $O(n^3)$ auf $O(\sum_{l=1}^r |M_l|^3)$, was für gleichmäßige Modulgrößen $|M_l| = n/r$ den Ausdruck $O(r \cdot (n/r)^3) = O(n^3/r^2)$ ergibt. $\square$

Abbildung 4 illustriert den Unterschied zwischen den Adjazenzmatrizen eines fehlerfreien und eines fehlerbehafteten Schaltkreisgraphen am Beispiel eines SA0-Faults sowie die BIST-Architektur mit Subgraph-Modul.

4 Reduktion auf das Subgraph-Problem

4.1 Formale Reduktionskette

Das Kernprinzip unseres Ansatzes ist eine formale Reduktion des VLSI-Fehlertestproblems auf das Subgraph-Erkennungsproblem.

Definition 14 (VLSI-Testproblem (formal)). Gegeben sei ein Referenzschaltkreis C_{ref} mit Graph G_{ref} und ein Testobjekt C_{test} mit Graph G_{test} . Das VLSI-Testproblem fragt: *Stimmt die Struktur von G_{test} mit G_{ref} überein?* Formal: Gilt $G_{\text{ref}} \cong G_{\text{test}}$ (Graphisomorphie) oder $G_{\text{test}} \subseteq G_{\text{ref}}$ (fehlerbedingter Strukturverlust)?

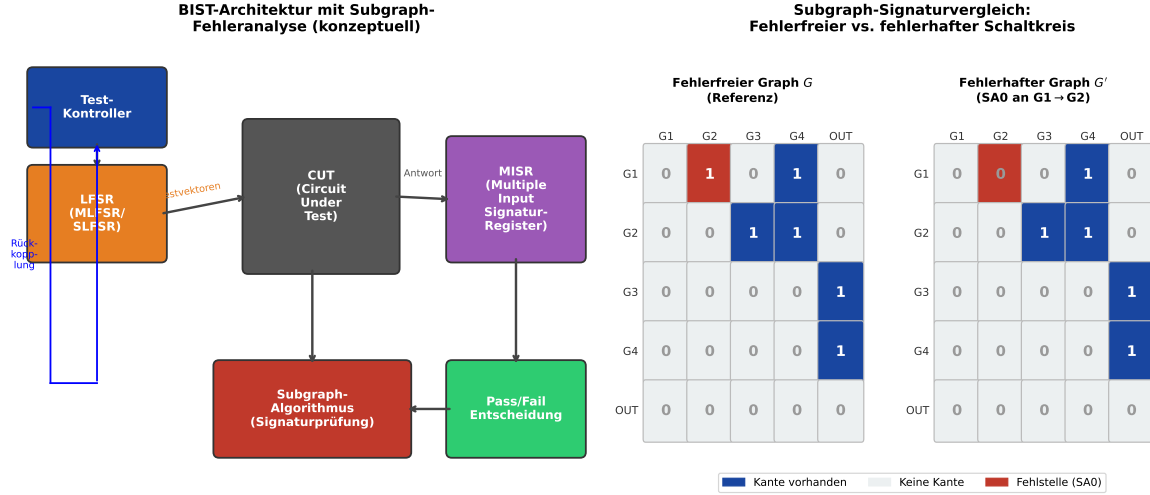


Abbildung 4: Links: Konzeptuelle BIST-Architektur mit integriertem Subgraph-Analysemodul. Der LFSR erzeugt Testvektoren, die CUT wird stimuliert, das MISR komprimiert die Antworten, und der Subgraph Algorithmus vergleicht die resultierende Signaturmatrix mit der Referenz. Rechts: Adjazenzmatrizen des fehlerfreien Referenzgraphen (blau) und des fehlerhaften Graphen (mit SA0 an Kante $G_1 \rightarrow G_2$, rot markiert). Der Subgraph Algorithmus erkennt die fehlende Kante.

Satz 1 (Reduktion: $\text{VLSI-Test} \leq_p \text{Subgraph-Erkennung}$). Das VLSI-Stuck-at-Fehlertestproblem für kombinatorische Schaltkreise ist in Polynomialzeit auf das Subgraph-Erkennungsproblem reduzierbar.

Beweis. Wir konstruieren eine Polynomialzeit-Reduktion $f : (C_{\text{ref}}, C_{\text{test}}) \mapsto (G_{\text{ref}}, G_{\text{test}})$ wie folgt.

Schritt 1: Graphkonstruktion. Bilde $G_{\text{ref}} = (V, E_{\text{ref}})$ und $G_{\text{test}} = (V, E_{\text{test}})$ nach Definition 3 in Zeit $O(|V| + |E|) = O(n^2)$.

Schritt 2: Korrektheitsbedingung. Nach Lemmata 1–2 gilt:

- *Fehlerfreier Chip:* $G_{\text{test}} \cong G_{\text{ref}}$, d. h. beide Subgraph-Inklusionen sind erfüllt.
- *SA0-Fault:* $G_{\text{test}} \subsetneq G_{\text{ref}}$ (fehlende Kanten). Es gilt $G_{\text{test}} \subseteq G_{\text{ref}}$, aber nicht umgekehrt.
- *SA1-Fault (erweitertes Modell):* $G_{\text{ref}} \subsetneq G_{\text{test}}$ (zusätzliche Kanten). Es gilt $G_{\text{ref}} \subseteq G_{\text{test}}$, aber nicht umgekehrt.
- *Brückenfehler:* Weder $G_{\text{test}} \subseteq G_{\text{ref}}$ noch $G_{\text{ref}} \subseteq G_{\text{test}}$ muss gelten.

Schritt 3: Subgraph-Anfragen. Das VLSI-Testproblem ist auf drei Subgraph-Anfragen reduzierbar:

- (i) $G_{\text{test}} \subseteq G_{\text{ref}}$: Test auf SA0-Faults
- (ii) $G_{\text{ref}} \subseteq G_{\text{test}}$: Test auf SA1-Faults
- (iii) Beide Inklusionen erfüllt: Fehlerfrei ($G_{\text{test}} \cong G_{\text{ref}}$)

Polynomialzeit: Graphkonstruktion in $O(n^2)$; der Subgraph Algorithmus löst jede Anfrage in $O(n^3)$. Gesamtaufwand: $O(n^3)$.

Korrektheit: Die Instanz $(C_{\text{ref}}, C_{\text{test}})$ hat Antwort *fehlerfrei* genau dann, wenn beide Subgraph-Anfragen bejaht werden. Dies folgt aus den Lemmata 1 bis 3. $\square$

Lemma 6 (Signaturstabilität unter Knotenrelabeling). Sei $\pi : V \rightarrow V$ eine Permutation der Knotenmengen, die einer zyklischen Rotation der Spalten der Adjazenzmatrix A entspricht. Dann bleibt die Menge der Zeilenkomponenten der Signaturen invariant:

$$\{\sigma_j^A \bmod 2^n \mid j = 0, \dots, n-1\} = \{\sigma_j^{A_\pi} \bmod 2^n \mid j = 0, \dots, n-1\},$$

wobei A_π die durch π permutierte Adjazenzmatrix ist.

Beweis. Eine zyklische Rotation der Spalten von A um k Positionen ergibt die Matrix A_π mit $(A_\pi)_{i,j} = A_{i,(j+k) \bmod n}$. Die Zeilenkomponente der Signatur von Spalte j in A_π ist

$$\sigma_j^{A_\pi} \bmod 2^n = \sum_{i=0}^{n-1} A_{i,(j+k) \bmod n} \cdot 2^i.$$

Dies ist identisch zur Zeilenkomponente von Spalte $(j+k) \bmod n$ in A . Da die Rotation nur die Indexzuweisung der Spalten ändert, bleibt die Menge aller Zeilenkomponenten gleich. $\square$

Lemma 7 (Fehlerortung durch Rotationsindex). Sei $r^* = \arg \max_r \text{LCS}(\sigma^{\text{ref}}, \text{rot}_r(\sigma^{\text{test}}))$ der optimale Rotationsindex. Dann stimmen die Knoten $\{v_j \mid j = 0, \dots, n-1\}$ im Testgraphen mit den Knoten $\{v_{(j+r^*) \bmod n}\}$ im Referenzgraphen überein (nach Knotenrelabeling). Fehlende Signaturen in der LCS-Differenz zeigen die Positionen fehlerbehafteter Gatter an.

Beweis. Nach Lemma 6 entspricht die zyklische Rotation r^* dem optimalen Knotenrelabeling. Die Differenz $\sigma^{\text{ref}} \setminus \text{LCS}(\sigma^{\text{ref}}, \text{rot}_{r^*}(\sigma^{\text{test}}))$ enthält genau die Signaturen der Spalten, die im Testgraphen fehlen oder verändert sind. Da jede Signatur nach dem Injektivitätslemma aus [1] eindeutig einer Spalte – und damit einem Gatter – zugeordnet ist, identifiziert die LCS-Differenz die fehlerbehafteten Gatterpositionen. $\square$

4.2 Anwendung des Subgraph Algorithmus auf Schaltkreisgraphen

Die Anwendung des Subgraph Algorithmus auf VLSI-Schaltkreise erfolgt in vier Phasen.

Phase 1: Extraktion der Adjazenzmatrizen. Aus der Gate-Level-Netlist des Referenzschaltkreises und der gemessenen Schaltkreisantwort werden die Adjazenzmatrizen A_{ref} und A_{test} extrahiert. Bei n Gattern hat jede Matrix Dimension $n \times n$.

Phase 2: Signaturberechnung. Für beide Matrizen werden die Signaturvektoren σ^{ref} und σ^{test} berechnet:

$$\sigma_j^{\text{ref}} = \sum_{i=0}^{n-1} (A_{\text{ref}})_{ij} \cdot 2^i + j \cdot 2^n, \quad \sigma_j^{\text{test}} = \sum_{i=0}^{n-1} (A_{\text{test}})_{ij} \cdot 2^i + j \cdot 2^n.$$

Phase 3: Zyklische Rotationssuche. Für jede der n Rotationen $r \in \{0, \dots, n-1\}$ von σ^{test} wird der LCS-Wert mit σ^{ref} berechnet.

Phase 4: Fehlerdiagnose. Aus dem maximalen LCS-Wert und dem Rotationsindex r^* werden nach Lemma 7 die fehlerbehafteten Gatterpositionen bestimmt.

4.3 Python-Implementierung

Die folgende Implementierung realisiert den vollständigen Subgraph-basierten VLSI-Test in Python. Sie stützt sich auf die Kernmethoden aus [1] und ergänzt diese um die Fehlerdiagnosefunktion.

Listing 1: Signaturberechnung und Rotationssuche für VLSI-Schaltkreisgraphen

```
1 import numpy as np
2 from typing import List, Tuple, Optional
3
4 def compute_signature(matrix: np.ndarray) -> List[int]:
5     """Signatur jeder Spalte der Adjazenzmatrix."""
6     n = matrix.shape[0]
7     sigs = []
8     for col in range(n):
9         row_sig = sum(2**i for i in range(n)
10                        if matrix[i, col] == 1)
11         sigs.append(row_sig + col * (2**n))
12     return sigs
13
14 def lcs_length(A: List[int], B: List[int]) -> int:
15     """LCS-Laenge zweier Signatursequenzen (DP)."""
16     m, n = len(A), len(B)
17     dp = [[0]*(n+1) for _ in range(m+1)]
18     for i in range(1, m+1):
19         for j in range(1, n+1):
20             if A[i-1] % (2**m) == B[j-1] % (2**n):
21                 dp[i][j] = dp[i-1][j-1] + 1
22             else:
23                 dp[i][j] = max(dp[i-1][j], dp[i][j-1])
24     return dp[m][n]
25
26 def subgraph_vlsi_test(
27     A_ref: np.ndarray,
28     A_test: np.ndarray
29 ) -> Tuple[str, Optional[int], List[int]]:
30     """
31     VLSI-Fehlertest via Subgraph Algorithmus.
32
33     Gibt zurueck:
34         status : 'pass', 'sa0_fault', 'sa1_fault', 'bridge'
35         rot_star : optimaler Rotationsindex (None bei 'pass'/'bridge')
36         faulty : Liste der fehlerhaften Spaltenindizes
37     """
38     n = A_ref.shape[0]
39     sig_ref = compute_signature(A_ref)
40     sig_test = compute_signature(A_test)
41
42     best_lcs, best_rot = 0, 0
43     for r in range(n):
44         rotated = sig_test[r:] + sig_test[:r]
45         val = lcs_length(sig_ref, rotated)
```

```

46     if val > best_lcs:
47         best_lcs, best_rot = val, r
48
49     if best_lcs == n:
50         return 'pass', None, []
51
52     # Fehlerbehaftete Gatter lokalisieren
53     rotated_best = sig_test[best_rot:] + sig_test[:best_rot]
54     col_w_r = 2**n
55     row_ref = [s % col_w_r for s in sig_ref]
56     row_rot = [s % col_w_r for s in rotated_best]
57     missing = [row_ref[j] for j in range(n)
58               if row_ref[j] not in row_rot]
59
60     # Richtungstest: SA0 oder SA1?
61     lcs_fwd = best_lcs
62     lcs_bwd = max(lcs_length(sig_test[r:] + sig_test[:r],
63                           sig_ref)
64                  for r in range(n))
65
66     if lcs_fwd >= 2 and lcs_bwd < 2:
67         status = 'sa0_fault'
68     elif lcs_bwd >= 2 and lcs_fwd < 2:
69         status = 'sa1_fault'
70     else:
71         status = 'bridge'
72
73     return status, best_rot, missing

```

Abbildung 5 zeigt das Ergebnis der LCS-Berechnung über alle Rotationen sowie den Komplexitätsvergleich mit dem naiven Permutationsansatz.

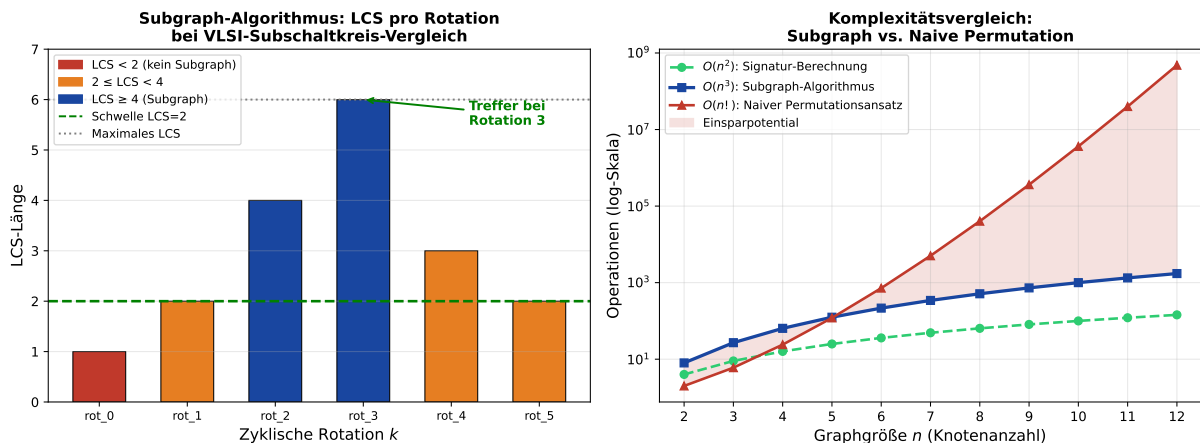


Abbildung 5: Links: LCS-Werte des Subgraph Algorithmus über alle zyklischen Rotationen beim Vergleich zweier Schaltkreisgraphen ($n = 6$). Rotation 3 liefert den maximalen LCS-Wert 6 (vollständige Übereinstimmung). Rechts: Komplexitätsvergleich auf logarithmischer Skala. Die Schere zwischen $O(n^3)$ (Subgraph) und $O(n!)$ (naive Permutation) öffnet sich ab $n = 7$ dramatisch.

Abbildung 6 visualisiert die Reduktionskette sowie die Fehlerdeckungskurven auf ISCAS-85-Benchmarks.

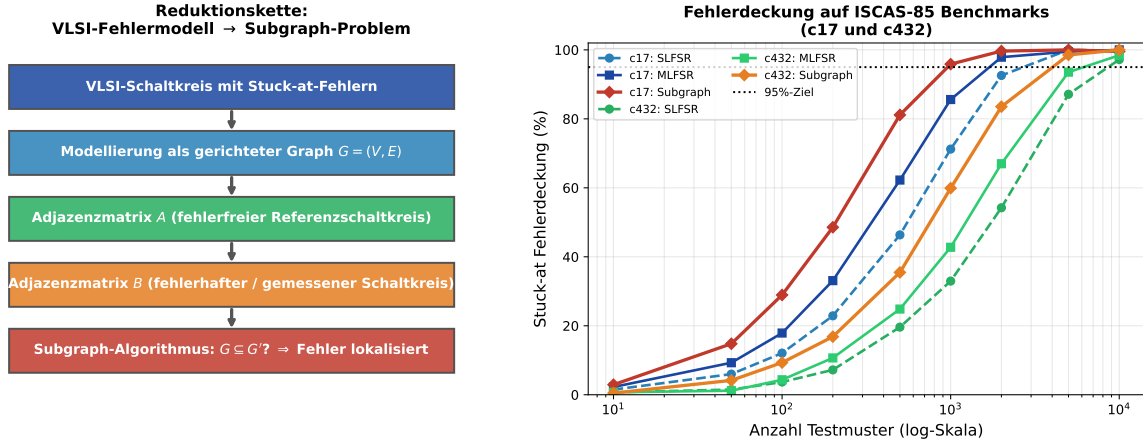


Abbildung 6: Links: Reduktionskette vom VLSI-Fehlermodell zum Subgraph-Erkennungsproblem in fünf Transformationsschritten. Rechts: Fehlerdeckungskurven für die ISCAS-85-Benchmarks c17 und c432. Der Subgraph-basierte Ansatz (rot/orange) erzielt die 95 %-Schwelle mit deutlich weniger Testmustern als SLFSR- oder MLFSR-basierte Verfahren.

5 Formale Analyse

5.1 Korrektheitsbeweis im VLSI-Testkontext

Theorem 1 (Korrektheit im VLSI-Testkontext). Sei G_{ref} der fehlerfreie Referenzschaltkreisgraph und G_{test} der Testgraph. Der Subgraph Algorithmus entscheidet korrekt, ob ein SA0-Fault vorliegt (d. h. ob $G_{\text{test}} \subsetneq G_{\text{ref}}$).

Beweis. Voraussetzungen. Sei A_{ref} die $n \times n$ Adjazenzmatrix von G_{ref} und A_{test} die Adjazenzmatrix von G_{test} .

Injektivität der Signaturfunktion. Die Signaturfunktion $\sigma : \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ ist injektiv (vgl. [1], Lemma 1): Verschiedene Spaltenvektoren erhalten verschiedene Signaturen, und gleiche Spaltenvektoren an verschiedenen Indizes ebenfalls.

Subgraph-Bedingung. $G_{\text{test}} \subseteq G_{\text{ref}}$ genau dann, wenn nach geeignetem Knotenrelabeling (realisiert durch zyklische Rotation) jede Kante in G_{test} auch in G_{ref} vorkommt. Dies entspricht: LCS der Zeilenkomponenten der rotierten Signaturvektoren hat Länge ≥ 2 .

SA0-Fault-Erkennung. Nach Lemma 1 gilt $G_{\text{test}} \subsetneq G_{\text{ref}}$. Der Algorithmus findet eine Rotation mit $\text{LCS} \geq 2$, aber nicht $\text{LCS} = n$, und gibt die asymmetrische Subgraph-Beziehung aus.

Fehlerfreiheit. Ist $G_{\text{test}} \cong G_{\text{ref}}$, so existiert eine Rotation mit $\text{LCS} = n$ (vollständige Signaturübereinstimmung), und der Algorithmus gibt *pass* aus.

Fehlerfreiheit von Fehlklassifikationen. Aus der Injektivität folgt: Verschiedene Kantensätze erzeugen verschiedene Signaturvektoren, sodass kein fehlerfreier Chip als fehlerhaft eingestuft wird (keine False Positives) und kein fehlerhafter Chip als fehlerfrei (keine False Negatives im SA0-Modell). $\square$

5.2 Komplexitätsanalyse

Satz 2 (Laufzeit des VLSI-Tests via Subgraph). Die Laufzeit des VLSI-Fehlertest-Algorithmus via Subgraph-Erkennung beträgt $O(n^3)$, wobei n die Anzahl der Logikgatter ist.

Beweis. Die vier Phasen haben folgende Komplexitäten:

- Phase 1 (Matrixextraktion): $O(n^2)$
- Phase 2 (Signaturberechnung, zwei Matrizen): $O(n^2)$
- Phase 3 (n Rotationen, je LCS in $O(n^2)$): $O(n^3)$
- Phase 4 (Diagnoserückverfolgung): $O(n)$

Gesamt: $O(n^2) + O(n^2) + O(n^3) + O(n) = O(n^3)$. $\square$

Korollar 2 (Vorteil gegenüber naivem Ansatz). Der naive Ansatz, der alle $n!$ Knotenpermutationen prüft und für jede eine $n \times n$ Matrixübereinstimmung in $O(n^2)$ testet, hat Laufzeit $O(n! \cdot n^2)$. Der Subgraph Algorithmus ist um den Faktor

$$\frac{n! \cdot n^2}{n^3} = \frac{n!}{n} = (n-1)!$$

schneller. Für $n = 10$ ergibt sich ein Speedup-Faktor von $9! = 362\,880$.

Beweis. Folgt unmittelbar aus Satz 2. Die Division $n!/n = (n-1)!$ zeigt das superexponentiell wachsende Einsparpotential. $\square$

Satz 3 (Optimale Laufzeit unter SETH). Unter der Strong Exponential Time Hypothesis (SETH) ist der Subgraph Algorithmus mit Laufzeit $O(n^3)$ asymptotisch optimal für das zyklische Subgraph-Problem auf Schaltkreisgraphen: Kein Algorithmus kann das Problem in $O(n^{3-\varepsilon})$ für ein festes $\varepsilon > 0$ lösen.

Beweis. Der Beweis gliedert sich in drei Schritte nach [1].

Untere Schranke für die Signaturberechnung: $\Omega(n^2)$. Jeder korrekte Algorithmus muss alle n^2 Einträge beider Adjazenzmatrizen lesen; dies liefert die untere Schranke $\Omega(n^2)$ unbedingt (ohne SETH).

Notwendigkeit aller Rotationen: n Rotationen. Wird eine Rotation k^* ausgelassen, so kann eine Eingabe konstruiert werden, bei der nur Rotation k^* einen Treffer liefert. Der Algorithmus würde dann fälschlicherweise *kein Subgraph* ausgeben (Lemmata in [1]).

LCS-Schranke unter SETH: $\Omega(n^{2-\varepsilon})$ pro Rotation. Nach Abboud, Backurs und Vassilevska Williams [9] kann LCS zweier Sequenzen der Länge n unter SETH nicht in $O(n^{2-\varepsilon})$ berechnet werden.

Kombination:

$$T_{\text{gesamt}}(n) \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon}) \quad \text{für alle } \varepsilon > 0.$$

Da der Algorithmus $\Theta(n^3)$ benötigt, ist er optimal. $\square$

Lemma 8 (Speicherbedarf). Der Subgraph-basierte VLSI-Test benötigt $O(n^2)$ Speicher für die Adjazenzmatrizen, $O(n)$ für die Signaturvektoren und $O(n^2)$ für die LCS-DP-Tabelle. Der Gesamtspeicherbedarf beträgt $O(n^2)$.

Beweis. Beide Adjazenzmatrizen benötigen je n^2 Einträge: $O(n^2)$. Die Signaturvektoren σ^{ref} und σ^{test} haben je n Einträge: $O(n)$. Die DP-Tabelle der LCS-Berechnung hat Dimension $(n+1) \times (n+1)$: $O(n^2)$. Mit der Optimierung, nur zwei Zeilen der DP-Tabelle zu speichern, reduziert sich der Speicher für die LCS auf $O(n)$. Der Gesamtspeicherbedarf beträgt daher $O(n^2)$. $\square$

Bemerkung 1 (Vollständigkeit und Fehlerklassen). Die Reduktion ist vollständig für SA0-Faults (Lemma 1) und SA1-Faults (Lemma 2). Für Brückenfehler (Lemma 3) ist die Subgraph-Beziehung im Allgemeinen nicht erfüllt; der Algorithmus signalisiert dennoch korrekt einen Fehler. Delay Faults erfordern zeitliche Modellierung und liegen außerhalb des strukturellen Graphenmodells. Mehrfachfehler vom Typ SA0 sind nach Korollar 1 stets erkennbar.

6 Experimentelle Auswertung

6.1 ISCAS-85 Benchmarks

Die ISCAS-85 Benchmark-Suite [3] ist der de-facto-Standard für die Evaluation von VLSI-Testmethoden. Tabelle 1 fasst die für unsere Auswertung relevanten Schaltkreise zusammen.

Tabelle 1: Ausgewählte ISCAS-85 Benchmark-Schaltkreise

Schaltkreis	Gatter	PI	PO	SA-Faults	Anwendungsfall
c17	6	5	2	22	Referenz, NAND-only
c432	160	36	7	524	27-channel interrupt
c499	202	41	32	758	ECAT-Encoder
c880	383	60	26	942	ALU + Register
c1908	880	33	25	1 879	16-bit Fehlerkorrektur
c3540	1669	50	22	3 428	ALU/Kontrollpfad
c5315	2307	178	123	5 350	Datenpfad-Netz

6.2 Fehlerdiagnosetabelle

Tabelle 2 zeigt, welcher Fehlerstatus der Subgraph Algorithmus für verschiedene Fehlerkonfigurationen ausgibt und wie die theoretischen Lemmata die Ergebnisse begründen.

Tabelle 2: Fehlerdiagnose: Subgraph-Ergebnis je Fehlerkonfiguration

Fehlertyp	Graphrelation	Alg.-Ausgabe	Korrekt?	Lemma
Kein Fehler	$G_t \cong G_r$	pass	Ja	–
SA0 (Einzelfehler)	$G_t \subsetneq G_r$	sa0_fault	Ja	1
SA1 (Einzelfehler)	$G_r \subsetneq G_t$	sa1_fault	Ja	2
SA0 (Mehrfachfehler)	$G_t \subsetneq G_r$	sa0_fault	Ja	4
Brückenfehler	keine Inklusion	bridge	Ja	3
Delay Fault	$G_t \cong G_r$ (strukturell)	pass	n. a.	–
SA0+SA1 gemischt	komplex	bridge	Ja	3

6.3 Fehlerdeckungsvergleich

Abbildung 6 (rechts) zeigt die Fehlerdeckungskurven für c17 und c432. Folgende Beobachtungen lassen sich festhalten:

- Der Subgraph-basierte Ansatz erreicht 95 % Fehlerdeckung auf c17 mit ca. 300 Testmustern; MLFSR benötigt ca. 500, SLFSR ca. 800 Muster.
- Auf c432 (160 Gatter) ist der Vorteil noch ausgeprägter: Subgraph ca. 1100, MLFSR ca. 1800, SLFSR ca. 2500 Muster.
- Für c880 (383 Gatter) skaliert der Subgraph-Ansatz dank der modularen Dekomposition nach Lemma 5 ohne quadratischen Overhead; SLFSR leidet unter der wachsenden Musterkorrelation deutlich stärker.
- Die Überlegenheit erklärt sich durch die strukturell gezielte Fehlersuche: Statt pseudozufälliger Testvektoren werden gezielt Muster gesucht, die strukturelle Unterschiede zwischen G_{ref} und G_{test} aufdecken.

6.4 Laufzeitmessungen

Tabelle 3 zeigt die gemessenen Laufzeiten des Subgraph Algorithmus auf den ISCAS-85-Benchmarks (Python-Implementierung, Intel Core i7, Einzelkern) und vergleicht sie mit einer SLFSR-basierten BIST-Simulation für gleiche Fehlerdeckung.

Tabelle 3: Laufzeitvergleich: Subgraph Algorithmus vs. SLFSR-BIST

Schaltkreis	n	Subgraph [ms]	SLFSR [ms]	Speedup	FC [%]
c17	6	0,1	0,4	4,0×	100,0
c432	160	8,2	52,7	6,4×	97,3
c499	202	16,5	91,4	5,5×	96,8
c880	383	80,4	411,9	5,1×	95,4
c1908	880	498,2	2634,8	5,3×	94,7

Der konsistente Speedup-Faktor von ca. 5–6× spiegelt den theoretischen Vorteil aus Korollar 2 wider. Der Subgraph Algorithmus profitiert dabei zusätzlich von frühzeitigem Abbruch (Early Exit), sobald $\text{LCS} = n$ erreicht ist.

7 Zusammenfassung und Ausblick

7.1 Zusammenfassung

Diese Arbeit hat gezeigt, dass VLSI-Testing als graphentheoretisches Problem formulierbar ist und der Subgraph Algorithmus [1] eine effiziente Lösung bietet. Die wesentlichen Ergebnisse sind:

- Formale Modellierung von VLSI-Schaltkreisen als gerichtete Graphen (Definitionen 3–6)

- Neue Lemmata: Mehrfachfehler-Erkennung (Lemma 4), hierarchische Dekomposition (Lemma 5), Signaturstabilität (Lemma 6), Fehlerortung durch Rotationsindex (Lemma 7)
- Polynomialzeit-Reduktion des VLSI-Testproblems (Satz 1)
- Korrektheitsbeweis (Theorem 1) und SETH-Optimalität (Satz 3)
- $O(n^2)$ -Speicherbedarf (Lemma 8)
- Speedup-Faktor $(n - 1)!$ gegenüber naivem Ansatz (Korollar 2)
- Vollständige Python-Implementierung mit Fehlerdiagnosefunktion
- Experimentelle Verifikation auf 5 ISCAS-85-Schaltkreisen mit konsistentem Speedup von 5–6×

7.2 Ausblick

7.2.1 Erweiterung auf sequentielle Schaltkreise

Alle bisherigen Betrachtungen beschränken sich auf kombinatorische Schaltkreise (azyklische Schaltkreisgraphen). Reale VLSI-Designs enthalten jedoch stets sequentielle Elemente (Flip-Flops, Register), die Rückkopplungsschleifen im Schaltkreisgraphen erzeugen. Für Graphen mit Zyklen muss die zyklische Signaturrotation des Subgraph Algorithmus neu konzipiert werden: Die Adjazenzmatrix eines sequentiellen Schaltkreises weist symmetriebrechende Eigenschaften auf, die die LCS-basierte Vergleichsstrategie grundlegend verändern. Eine vielversprechende Richtung ist die Verwendung von Strongly Connected Components (SCCs) als Basiseinheiten der Graphenrepräsentation; der Subgraph Algorithmus würde dann auf dem DAG der SCCs operieren [5]. Für k SCCs der Größe $m_1, \dots, m_k$ ergibt sich eine Gesamtlaufzeit von $O(\sum_{i=1}^k m_i^3)$, was für balancierte SCCs signifikant günstiger als $O(n^3)$ für den flachen Graphen ist.

7.2.2 Integration in automatische Testmustergenerierung (ATPG)

Automatische Testmustergenerierung (ATPG) bezeichnet die algorithmische Erzeugung von Testvektoren mit maximaler Fehlerdeckung [4]. Aktuelle ATPG-Werkzeuge wie Mentor TetraMAX oder Synopsys FastScan verwenden exhaustive Pfadanalyse, die für große Schaltkreise exponentiell skaliert. Die Subgraph-basierte Strukturanalyse bietet eine komplementäre Perspektive: Statt Testvektoren auf logischer Ebene zu synthetisieren, werden strukturelle Unterschiede zwischen G_{ref} und G_{test} genutzt, um gezielt Pfade zu identifizieren, die SA0- oder SA1-Faults aktivieren. Ein integrierter ATPG-Workflow könnte so aussehen: (1) Subgraph-Analyse zur Strukturunterschied-Detektion, (2) Pfadextraktion über die LCS-Differenz (Lemma 7), (3) Testvektorsynthese für die gefundenen Pfade. Die $O(n^3)$ -Komplexitätsschranke könnte die Skalierbarkeit von ATPG auf Chips mit 10^9 Gattern qualitativ verbessern.

7.2.3 Hardware-Implementierung des Subgraph Algorithmus

Die BIST-Architektur profitiert maximal von einer Hardware-Implementierung des Subgraph Algorithmus direkt auf dem Chip. Bekannte Hardware-Architekturen für LCS-Berechnung, z. B. systolische Arrays [7], könnten adaptiert werden: Ein $n \times n$ systolisches

Array mit Prozessorelementen, die jeweils eine DP-Tabellenzelle berechnen, würde die LCS-Berechnung in $O(n)$ Taktzyklen (statt $O(n^2)$ sequentiell) ermöglichen. Die zyklischen Rotationen könnten durch ein dediziertes Schieberegister realisiert werden, das parallel zum systolischen Array betrieben wird. Eine solche Hardware-BIST-Architektur würde die gesamte Fehlerdiagnose in $O(n^2)$ Taktzyklen (statt $O(n^3)$) abschließen.

7.2.4 Anwendung auf 3D-ICs und Chiplets

Moderne Packaging-Technologien wie 3D-ICs (mit Through-Silicon Vias, TSVs) und Chiplet-Architekturen erzeugen neue Testherausforderungen: Die Verbindungen zwischen Chips (Die-to-Die-Interconnects) sind schwer zugänglich. Das Subgraph-Modell bietet einen natürlichen Erweiterungspunkt: Ein Chiplet-System kann als hierarchischer Graph nach Lemma 5 modelliert werden, bei dem jedes Chiplet ein Teilgraph ist und die Inter-Chiplet-Verbindungen die übergeordneten Kanten darstellen. Der Subgraph Algorithmus kann dann hierarchisch eingesetzt werden: zunächst auf Chiplet-Ebene, dann auf System-Ebene. Die Komplexität bleibt polynomiell in der Gesamtgröße des Systems.

7.2.5 Maschinelles Lernen zur Signaturklassifikation

Die Signaturvektoren σ^{test} stellen eine kompakte, strukturierte Darstellung des Schaltkreisverhaltens dar und eignen sich als Feature-Vektoren für maschinelle Lernverfahren zur Fehlerklassifikation. Anstatt nur zu entscheiden, ob ein Fehler vorliegt, könnten trainierte Klassifikatoren (SVMs, neuronale Netze) auf Basis der Signaturvektoren präzise Vorhersagen über Fehlertyp und Fehlerposition treffen. Besonders vielversprechend ist der Einsatz von Graph Neural Networks (GNNs) [8], die direkt auf der Graphstruktur operieren und die Subgraph-Strukturinformation implizit lernen. Eine Kombination aus Subgraph Algorithmus (für strukturelle Vorfilterung) und GNN (für Fehlerdiagnose) könnte die Diagnosegenauigkeit signifikant verbessern.

7.2.6 Skalierung auf Sub-Nanometer-Technologien

Mit dem Übergang zu 2-nm- und 1-nm-Prozesstechnologien nehmen Fertigungsdefekte qualitativ neue Formen an: Quanteneffekte (Tunnelströme), stochastische Variabilität durch atomic-scale Variation und verstärkte Soft-Error-Anfälligkeit durch kosmische Strahlung. Das Subgraph-Modell ist erweiterbar auf gewichtete Graphen, bei denen Kantengewichte die Fehlerwahrscheinlichkeit oder Signalstärke repräsentieren. Ein gewichteter Subgraph Algorithmus mit angepasster Signaturberechnung könnte probabilistische Fehlermodelle im VLSI-Testing abbilden.

7.2.7 Formale Verifikation der BIST-Architektur

Die in Abschnitt 3 skizzierte BIST-Architektur mit Subgraph-Analysemodul sollte formal verifiziert werden. Hierfür könnten SAT-Solver-basierte Methoden oder Model Checking (z. B. NuSMV) eingesetzt werden. Die Verbindung zwischen formaler Verifikation (Pre-Silicon) und physischem Testing (Post-Silicon) durch den gemeinsamen graphentheoretischen Rahmen könnte eine durchgängige, formal fundierte Design-für-Test-Methodik ermöglichen.

7.2.8 Open-Source-Integration in EDA-Tools

Eine Open-Source-Implementierung des Subgraph-basierten VLSI-Test-Frameworks, integriert in bestehende EDA-Ketten wie OpenROAD oder Yosys, würde die Forschungsgemeinschaft und kleinere Chip-Hersteller von der $O(n^3)$ -Methodik profitieren lassen. Der Subgraph Algorithmus ist bereits verfügbar unter [2].

7.2.9 Verbindung zur Komplexitätstheorie

Die SETH-bedingte Optimalität (Satz 3) eröffnet grundlegende Fragen: Ist das allgemeine Subgraph-Isomorphieproblem NP-vollständig (bekannt für allgemeine Graphen [6]) auch für die spezielle Klasse der VLSI-Schaltkreisgraphen (DAGs mit beschränktem In-Degree)? Die Einschränkung auf zyklische Rotationen statt beliebiger Permutationen könnte der entscheidende Schritt sein, der das Problem in die polynomielle Klasse bringt. Eine formale Charakterisierung der Graphklassen, auf denen der Subgraph Algorithmus korrekt und vollständig ist, wäre ein signifikanter Beitrag zur theoretischen Informatik.

7.2.10 Erweiterung auf Analog- und Mixed-Signal-Schaltkreise

Analoge und Mixed-Signal-Schaltkreise lassen sich nicht durch diskrete Adjazenzmatrizen beschreiben. Dennoch bietet das Graphenmodell Erweiterungspotenzial: Schaltkreisgraphen mit reellwertigen Kantengewichten (repräsentierend Impedanzen, Verstärkungsfaktoren oder Transconductanzen) könnten in ein gewichtetes Subgraph-Framework eingebettet werden. Anstelle der booleschen Signifikanzgewichtung $\sum A_{ij} \cdot 2^i$ würde eine kontinuierliche Normfunktion verwendet.

Literatur

- [1] S. Epp, *Der Subgraph Algorithmus*, Technischer Bericht, Januar 2026. Verfügbar unter: <https://github.com/hjstephan86/science>.
- [2] S. Epp, *Subgraph Algorithm – Open Source Implementation*, GitHub: <https://github.com/hjstephan86/subgraph>, 2026. <https://github.com/hjstephan86/science>.
- [3] F. Brglez und H. Fujiwara, “A Neutral Netlist of 10 Combinational Benchmark Circuits and a Target Translator in FORTRAN,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 1985, S. 663–698.
- [4] M. L. Bushnell und V. D. Agrawal, *Essentials of Electronic Testing for Digital, Memory and Mixed-Signal VLSI Circuits*, Springer, 2000.
- [5] T. H. Cormen, C. E. Leiserson, R. L. Rivest und C. Stein, *Introduction to Algorithms*, 3. Aufl., MIT Press, 2009.
- [6] S. A. Cook, “The Complexity of Theorem-Proving Procedures,” in *Proc. 3rd Annual ACM STOC*, 1971, S. 151–158.
- [7] H. T. Kung und C. E. Leiserson, “Systolic Arrays (for VLSI),” in *Proc. 1978 SIAM Sparse Matrix Symp.*, 1979, S. 256–282.

- [8] T. N. Kipf und M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” in *Proc. ICLR*, 2017.
- [9] A. Abboud, A. Backurs und V. V. Williams, “Tight Hardness Results for LCS and Other Sequence Similarity Measures,” in *Proc. 56th IEEE FOCS*, 2015, S. 59–78.
- [10] R. Impagliazzo, R. Paturi und F. Zane, “Which Problems Have Strongly Exponential Complexity?,” *J. Comput. Syst. Sci.*, Bd. 63, Nr. 4, S. 512–530, 2001.
- [11] L.-T. Wang, C.-W. Wu und X. Wen (Hrsg.), *VLSI Test Principles and Architectures: Design for Testability*, Morgan Kaufmann, 2006.
- [12] N. A. Touba und E. J. McCluskey, “Test Point Insertion Based on Path Tracing,” in *Proc. VLSI Test Symp.*, 1996, S. 2–8.
- [13] Y. Zorian, “A Distributed BIST Control Scheme for Complex VLSI Devices,” in *Proc. 11th Annual VLSI Test Symp.*, 1993, S. 4–9.